

Final Report

Polonska Kristina, Mariana Konovalenko, Oksana Yefimova

September 27, 2026

1 Introduction

Our project of choice was American Express - Default Prediction.

2 Data Insights

2.1 Target and Customer History

The target variable is moderately imbalanced: 74.1% of customers belong to the non-default class and 25.9% to the default class.

Customer history length is also informative. Most customers have the maximum history of 13 statements, with 386 034 customers in this group. Their default rate is 23.2%, while customers with shorter histories generally show higher default rates. For example, customers with 10 statements have a default rate of 46.2%. This means that the number of statements may contain predictive information.

2.2 Missing Values

Volumes of NaNs is significant: 122 out of 190 features contain missing values. The analysis also showed that missingness is associated with the target for several variables.

For example, for `S_25`, customers with a missing value had a default rate of approximately 75.2%, compared with 25.7% when the feature was present. Similar differences were observed for several other features. Therefore, missing values should not necessarily be treated only as data-quality problems, since the fact that a value is missing is predictive information.

2.3 Predictive Features and Correlations

Several numerical features demonstrated strong individual ability to distinguish between defaulting and non-defaulting customers. The strongest features included `P_2`, `D_48`, `D_61`, `B_18`, `D_77`, `B_7`, and `D_75`.

At the same time, some of these features are strongly correlated. For example, `B_7` and `B_23` have a correlation of approximately 0.99, while `D_48` and `D_55` have a correlation of approximately 0.90. This indicates that some features contain similar information.

Several important numerical variables are also skewed or contain extreme values. For example, `D_61` has a skewness of approximately 5.95 and values extending far beyond its typical range.

2.4 Temporal Behaviour

Recent customer behaviour appears to be the most informative. For `P_2`, the mean over the latest three statements achieved a ROC-AUC of 0.9134, compared with 0.8995 for the mean over the customer's full history.

The latest observation was also highly predictive for many important features. However, historical summaries such as means, recent averages, standard deviations, and changes still contained additional information. This suggests that customer histories can be aggregated into one row per customer without relying only on the final observation.

2.5 Train vs Test Distribution Shift

The training and test datasets cover different time periods. Training observations range from March 2017 to March 2018, while test observations range from April 2018 to October 2019.

Differences were also found in feature distributions. The largest numerical shifts were observed for `S_11`, `D_59`, `B_39`, and `S_15`. Missing value patterns also changed between datasets. For example, `B_29` is missing in approximately 93.1% of training observations but 75.0% of test observations, while `S_9` decreases from 53.0% to 35.1%.

2.6 Adversarial Validation

Adversarial validation showed a shift between the training and test datasets. The train and test samples were combined into a single dataset, where training observations were assigned class 0 and test observations class 1. A logistic regression classifier was then trained to distinguish between the two datasets.

The classifier achieved a ROC-AUC of 0.7823. Since a ROC-AUC of 0.5 would mean that train and test are indistinguishable, this result shows that the two datasets differ in their feature distributions.

3 Feature Engineering

Categorical variables were identified using the aggregated `_last` features, while `customer.ID` and the target variable were excluded from the model inputs.

3.1 Numerical Features

Missing numerical values were replaced using median imputation. Median imputation was selected because it is less sensitive to extreme values than the mean.

After imputation, all numerical features were standardized using `StandardScaler`. Each feature was transformed to have approximately zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma}$$

Standardization was applied to ensure that numerical variables with different scales contribute more consistently during neural network training.

3.2 Categorical Features

Categorical features were represented by their latest observed values, identified by the `_last` suffix.

Missing categorical values were replaced using the most frequent category. Each categorical variable was then converted into integer labels using `LabelEncoder`.

For every categorical feature, the number of unique encoded categories was stored in `cat_dims`. These dimensions are later used to define embedding layers in the neural network.

3.3 Model Input Representation

After preprocessing, numerical and categorical features were stored separately.

Numerical features were converted to `float32`, while categorical features were converted to integer. The target variable was stored as a `float32`.

The final dataset therefore provides three model inputs:

- categorical feature tensor
- numerical feature tensor
- target tensor for training data

This separation allows the model to process numerical variables directly while categorical variables can be represented through embedding layers.

4 Models Tried

Histogram Gradient Boosting Classifier from sklearn library was used as a Machine Learning Baseline. This choice is explained by the model's faster speed for big datasets compared to standard GradientBoostingClassifier, as well as its native support for missing values. L2 regularisation was set to 0.5 in order to prevent overfitting by reducing noise and model's confidence towards it.

To explore deeper relationships within the data, a custom Deep Learning framework was implemented using PyTorch. For preprocessing and preventing data leakage, missed numerical features were replaced by the median then all scaled and missed categorical features were filled with constant string 'missing' since NaN itself plays as a distinct feature (was stated in EDA part). Categorical features were processed through parallel embedding layers, while numerical features were passed through a custom linear layer with L2 weight normalization. The network was optimized using a custom implementation of the AdamW algorithm, utilizing decoupled weight decay and a base learning rate of 0.001 for stable gradient updates. The model was trained for 10 epochs with a batch size of 512, ensuring smooth gradient estimations and optimal memory utilization.

Model	Main Hyperparameters
HistGradientBoostingClassifier	max_iter=150 learning_rate=0.1 l2_regularization=0.5 random_state=336
PyTorch Deep Learning Model	optimizer=CustomAdamW (based on AdamW, lr=0.001, weight_decay=1e-4) scheduler=OneCycleLR (max_lr=5e-3) batch_size=512 epochs=10 aux_weight=0.3

Table 1: Models and hyperparameters

5 Model Evaluation

To evaluate and compare the models, ROC-AUC score was chosen as the primary metric, which is standard for imbalanced financial default datasets. Stratified 5-Fold Cross-Validation was employed to ensure reliable out-of-fold (OOF) evaluations.

Model	Total OOF ROC-AUC	Average Fold AUC	Submission Final Score
HistGradientBoostingClassifier	0.9600	0.9599	0.79739
PyTorch Deep Learning Model	0.9606	0.9606	0.79895
Ensemble	0.9613	0.9613	0.80130

Table 2: Model performance

6 Optimization

For the HistGradientBoostingClassifier, optimization focused on preventing overfitting through L2 regularization and balancing the learning rate (0.1) with the number of iterations (150) to ensure smooth tree construction.

For the Deep Learning model, optimization was driven by structural and training-level techniques. The OneCycleLR scheduler dynamically adjusted the learning rate up to a peak of 5e-3, enabling super-convergence and helping the optimizer escape sharp local minima. Regularization was enforced via weight decay (1e-4) within a custom AdamW optimizer, alongside spatial dropout (0.2-0.4) and batch normalization to stabilize internal covariate shifts across layers.

7 Model Insights

ML Baseline: Permutation importance analysis on the HistGradientBoostingClassifier demonstrated that variables from the most recent statement month carry the highest predictive weight. The top 10 most influential features and their mean permutation importance scores are detailed in Table 3.

Deep Learning Model: The deep neural network successfully leveraged its parallel architecture. Embedding layers for categorical features mapped sparse codes into dense, meaningful vector spaces. Furthermore, the analysis of gradient flows confirmed the complete absence of vanishing gradients, validating the effectiveness of the Kaiming initialization and the auxiliary loss branch in promoting robust intermediate representations.

Feature	Importance Mean	Importance Std
P_2_last	0.017834	0.000248
B_1_last	0.002436	0.000139
B_4_last	0.001841	0.000023
D_39_last	0.001685	0.000061
D_42_mean	0.001254	0.000058
B_3_last	0.000875	0.000036
D_46_last	0.000619	0.000013
B_5_last	0.000517	0.000024
S_3_mean	0.000490	0.000063
D_41_last	0.000371	0.000047

Table 3: Top 10 features by permutation importance (HistGradientBoostingClassifier)

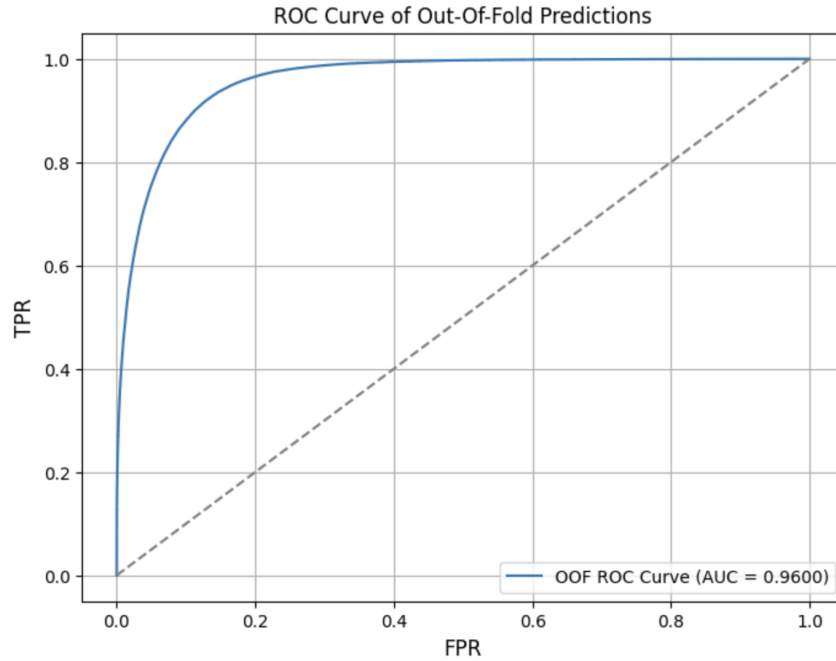


Figure 1: ROC-AUC Curve of OOF Predictions on HistGradientBoostingClassifier

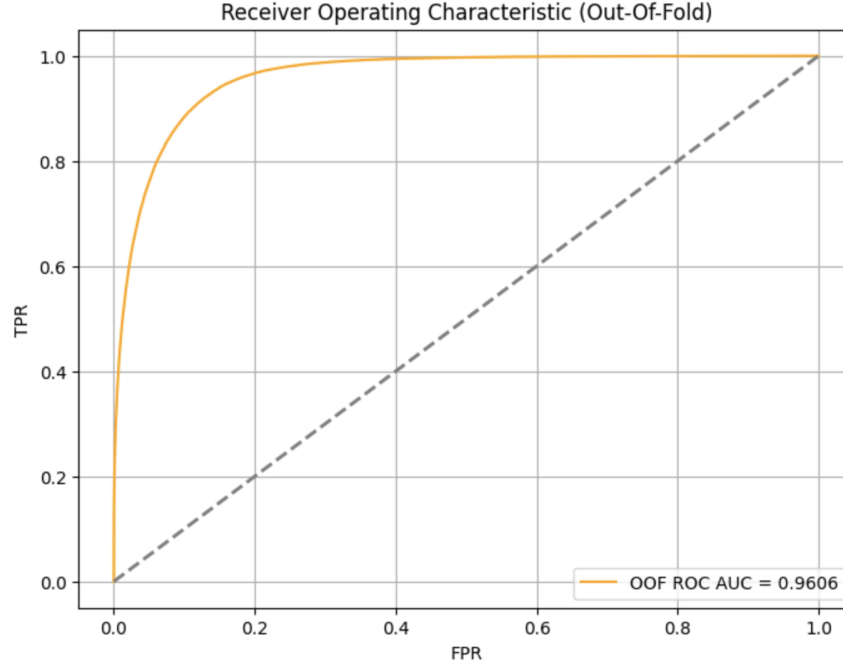


Figure 2: ROC-AUC Curve of OOF Predictions on Deep Learning Model

8 Conclusions

An ensemble achieved 0.9613 ROC-AUC by combining gradient boosting (0.9600 OOF) and deep learning (0.9606 OOF). Both models performed comparably, indicating overlapping information capture. Recent statement features dominated predictions, missing value patterns were predictive rather than data-quality issues. Deep learning embedding layers and gradient boosting’s native missing-value handling both proved effective, though the modest ensemble gain does not justify deep learning’s added complexity for this dataset.